IS1200 Advanced Project Report

Snail Run

Alex Ryström & Astrid Leonard

Fall 2025

Project Description

In this project, we have implemented and analyzed a real-time endless runner game called **Snail Run**, running on the DE10-Lite FPGA board which uses the RISC-V instruction set. The game renders live graphics to a VGA display and takes input from board switches and keys. The player moves between three lanes to avoid oncoming obstacles, and the game increases in difficulty over time as obstacles spawn and move faster. The objective of this report is to perform a detailed performance analysis of the project using the DTEK-V hardware performance counters.

Methods

To collect performance data, we used the methods described in the "Introduction to DTEK-V Hardware Performance Counters" and accessed the counters using inline assembly as seen in the snippet here:

```
// Reset counters
asm volatile("csrw mcycle, x0");
....rest of counters

// Read start values
asm volatile("csrr %0, mcycle" : "=r"(cycles_start));
....rest of counters
```

The counters measured include:

- `mcycle` – number of clock cycles elapsed
- `minstret` – number of instructions retired
- `mhpmcounter3--9` – memory instructions, cache misses, and stalls

For each measurement, we first cleared all the counters, then executed the target code (either an iteration of the game loop or specific functions within the loop), and finally read the counters. The values that were calculated in the code, consisting of IPC, cache miss ratios, etc. were collected for 1000 frames/calls and was printed to the terminal as an average using the given `print_dec` and `print` functions. We measured the full game loop as well as individual functions to understand which parts of the game contributed more to inefficiencies.

We analyzed three sections:

1. The game loop
2. The draw_game function
3. The update_game_state function

Additionally, to investigate the effect of compiler optimizations, we compiled and executed two versions of the game: (i) an unoptimized version with -O0 and (ii) an optimized version with -O3.

Results

Table 1 and 2 show the measured and derived metrics for the optimized and unoptimized versions of the program respectively.

	draw_game()	**update_game_state()**	**whole game loop**
IPC	0.52	0.11	0.61
Memory instr ratio	37%	35%	25%
I-cache hit rate	>99%	82%	>99%
D-cache hit rate	>99%	98%	>99%
I-cache stalls	<1%	79%	<1%
D-cache stalls	2%	15%	2%
Data Hazard stalls	15%	<1%	12%
ALU stalls	<1%	<1%	<1%

Table 1: Performance metrics for various sections of the program (optimized)

	draw_game()	**update_game_state()**	**whole game loop**
IPC	0.63	0.18	0.63
Memory instr ratio	47%	36%	49%
I-cache hit rate	>99%	91%	>99%
D-cache hit rate	>99%	98%	>99%
I-cache stalls	<1%	66%	<1%
D-cache stalls	13%	28%	12%
Data Hazard stalls	22%	2%	21%
ALU stalls	<1%	<1%	<1%

Table 2: Performance metrics for various sections of the program (unoptimized)

Discussion of IPC

The peak IPC is **0.63** achieved in the unoptimized version of the game loop, while the lowest observed IPC is **0.11** in the optimized `update_game_state()` function.

Performance Bottlenecks

The D-cache keeps a good hit rate (>97%). ALU stalls remain below 1%, confirming that the arithmetic units are not a major source of delay. The main limiting factor is the **I-cache performance**. In both the optimized and unoptimized compiled versions of the code, the I-cache hit rate drops quite low. This means that a number of the instructions the CPU needs are

not in the instruction cache, and they must be fetched from main memory, lowering IPC. This is likely due to the branch-heavy code featured in the function. Additional possible contributing factors include:

1. **Memory bottlenecks:** frequent reads/writes to the VGA frame buffer and game state exceed cache size, causing delays.
2. **Instruction mix:** the high number of control-flow instructions limits the number of instructions that can be executed per cycle.
3. **Cache capacity:** while the D-cache is mostly hit, large pieces of data like sprite data can still overflow the D-cache, causing occasional capacity misses.

Effect of Compiler Optimizations (-O0 vs. -O3)

It is a little unexpected that the optimized (`-O3`) version measured a lower IPC compared to the unoptimized (`-O0`) version, since compiler optimizations typically aim to increase instruction level parallelism and reduce stalls. However, a lower IPC does not necessarily imply worse performance. In this case, the optimized version executes far fewer total instructions (not included in table, but measured during testing), each performing more useful work, which reduces overall cycle count even if fewer instructions are retired per cycle on average. The unoptimized version, by contrast, could have been executing many redundant memory and stack operations that artificially inflate the IPC metric.

Although not included in our tables, there was a clearly visible difference in runtime performance. The optimized build rendered frames noticeably faster and provided a smoother gameplay experience. This demonstrates that IPC alone is not a complete indicator of performance.

Architectural Improvements

There are multiple ways to improve the processor architecture to make **Snail Run** run faster. The most impactful change would be to address the instruction cache limitations observed during profiling of the `update_game_state()` function. Increasing the instruction cache size would help mitigate the instruction fetch stalls seen, improving overall IPC. Another improvement would be to add a basic branch predictor to handle the frequent control-flow decisions in obstacle and collision detection. Additionally, introducing simple parallelism, such as a second core could allow object updates and pixel operations to run at the same time, reducing frame rendering time and making the gameplay smoother.